MEA / Neuralynx Spike Sorting GUI

Researcher Guide for Loading, QC, Sorting, Reports, Sorter Comparison, and Unit Curation

UI: MEA-Nlx Clustering UI_v9.ipynb + Core: MEA-Nlx Sorter python core_v6.py

1. Running environment and hardware expectations

This GUI can be run either on a computational core/HPC workstation or on a personal laptop/desktop. It is not restricted to the computational core. The main differences are expected runtime, available memory, available disk space, and GPU availability for Kilosort4.

For Neuralynx tetrode recordings, local laptop use is generally practical because each run sorts one tetrode group at a time. A tetrode recording typically contains four channels before QC, and fewer if the researcher excludes one or more noisy/broken wires. This is much lighter than sorting a full MEA recording.

For MEA .h5 recordings, local use depends more strongly on the number of channels, recording duration, number of detected units, and whether the researcher generates full reports and PC-based metrics. MEA sorting can still be run locally, but it may take longer and require more disk space because the workflow creates sorter outputs, waveform/analyzer folders, quality metric workbooks, spike-time exports, unit reports, curation workbooks, and optional comparison folders.

1.1 CPU parallelization

The GUI exposes worker-count settings for some sorter and postprocessing operations. On a computational core, higher worker counts can be useful. On a personal laptop, worker counts should usually be reduced to avoid making the machine unresponsive or causing memory pressure.

Computer type	Recommended starting worker count
4-core laptop	2 workers
8-core laptop	4 workers
High-end laptop	6-8 workers
Computational core	8-12+ workers, depending on available CPUs and memory

The UI also sets thread-related environment variables such as OMP_NUM_THREADS, MKL_NUM_THREADS, OPENBLAS_NUM_THREADS, and NUMEXPR_NUM_THREADS. These reduce oversubscription by preventing multiple numerical libraries from trying to use all CPU threads at the same time. This is useful on both laptops and computational cores.

1.2 GPU pinning

The UI includes GPU pinning with:

```
os.environ.setdefault("CUDA_DEVICE_ORDER", "PCI_BUS_ID")
os.environ.setdefault("CUDA_VISIBLE_DEVICES", "0")
```

This tells CUDA-aware libraries to use GPU 0 when a CUDA-compatible GPU is available. It does not force the computer to have a GPU, and it does not by itself prevent the GUI from running on a CPU-only machine.

MountainSort4 and SpyKING Circus2 can be used without a CUDA GPU if their dependencies are installed. Kilosort4 can also be installed and selected from the GUI, but it is designed as a GPU-accelerated spike sorter. On computers without a properly configured NVIDIA/CUDA setup, Kilosort4 may take longer, especially for larger recordings.

1.3 Sorter-specific expectations

Sorter	Practical expectation
MountainSort4	Usually practical for both local laptop testing and computational-core runs. For laptop use, reduce the worker count.
SpyKING Circus2	Implemented as an internal SpikeInterface sorter. It can be used locally, but runtime depends on recording size, channel count, and parameters.
Kilosort4	Installed as the Python package kilosort but selected in SpikeInterface as kilosort4. Performance depends strongly on PyTorch/CUDA setup. On non-GPU computers it may take longer and should be tested first on smaller recordings.

1.4 Disk-space expectations

The GUI writes multiple output folders and files. Researchers should use a local drive or high-speed external drive with sufficient free space. Network drives can work but may be slower, especially during waveform extraction and report generation.

```
si_sorting/
waveforms/
quality_metrics.xlsx
units_spike_times.xlsx
unit_reports/
curated_units_*.xlsx
sorter_comparison_* folders
```

The compare-sorters workflow is the most output-heavy because it runs two sorter pipelines, automatically generates spike times, quality metrics, unit reports for each sorter, and then creates agreement/matching outputs. For local laptop use, compare-sorters is best used for smaller test recordings, tetrode runs, or method validation.

1.5 Example environment check interpretation

A properly configured environment should show all required core packages as available and should report the relevant sorters as installed through SpikeInterface. A healthy check should look conceptually like:

```
Core GUI packages: OK

Installed sorters reported by SpikeInterface:
- mountainsort4
- kilosort4
- spykingcircus2
```

For Kilosort4, a GPU-capable environment should also show:

```
CUDA available: True
kilosort package import: OK
```

If CUDA is not available, the GUI can still run, but Kilosort4 will not have CUDA acceleration in that kernel.

2. Software installation and required packages

The GUI should be run inside a dedicated Python environment. A dedicated environment prevents conflicts with other Python projects and makes it easier to reproduce the sorter setup.

Item	Recommendation
Environment name	mea_nlx_sorter
Python version	Python 3.10 is recommended for new installs. Python 3.11 can work if all packages import and the sorters are detected.
Primary SpikeInterface version	SpikeInterface 0.103.0
Important install command	pip install "spikeinterface[full,widgets]==0.103.0"

macOS shell note: On macOS zsh, packages with bracketed extras must be quoted, for example "spikeinterface[full,widgets]==0.103.0" and "kilosort[gui]".

2.1 Windows installation: Anaconda Prompt

```
conda create -n mea_nlx_sorter python=3.10 -y
conda activate mea_nlx_sorter

python -m pip install --upgrade pip setuptools wheel

pip install "spikeinterface[full,widgets]==0.103.0"

pip install mountainsort4==1.0.6

pip install "kilosort[gui]"

pip install probeinterface==0.3.0 neo==0.14.2 quantities==0.16.2
```

```
pip install ipywidgets ipyfilechooser ipydatagrid ipykernel

pip install numpy==2.2.6 pandas==2.3.1 matplotlib==3.10.5 scipy==1.15.3 scikit-learn==1.7.1

pip install openpyxl==3.1.5 xlsxwriter==3.2.5 tqdm==4.67.1 joblib==1.5.1 h5py==3.14.0 pyyaml==6.0.2

pip install numba==0.61.2 llvmlite==0.44.0 psutil==5.9.0 matplotlib-inline==0.1.7

python -m ipykernel install --user --name mea_nlx_sorter --display-name "Python (MEA/NLX Sorter)"
```

Then launch Jupyter and select the Python (MEA/NLX Sorter) kernel:

```
jupyter lab
# or
jupyter notebook
```

2.2 macOS installation: Terminal

```
conda create -n mea_nlx_sorter python=3.10 -y
conda activate mea_nlx_sorter

python -m pip install --upgrade pip setuptools wheel

pip install "spikeinterface[full,widgets]==0.103.0"

pip install mountainsort4==1.0.6

pip install "kilosort[gui]"

pip install probeinterface==0.3.0 neo==0.14.2 quantities==0.16.2

pip install ipywidgets ipyfilechooser ipydatagrid ipykernel

pip install numpy==2.2.6 pandas==2.3.1 matplotlib==3.10.5 scipy==1.15.3 scikit-learn==1.7.1

pip install openpyxl==3.1.5 xlsxwriter==3.2.5 tqdm==4.67.1 joblib==1.5.1 h5py==3.14.0 pyyaml==6.0.2

pip install numba==0.61.2 llvmlite==0.44.0 psutil==5.9.0 matplotlib-inline==0.1.7

python -m ipykernel install --user --name mea_nlx_sorter --display-name "Python (MEA/NLX Sorter)"
```

Then launch Jupyter and select the Python (MEA/NLX Sorter) kernel:

```
jupyter lab
# or
jupyter notebook
```

2.3 Sorter installation notes

Sorter	Install/package note
MountainSort4	Install with pip install mountainsort4==1.0.6. If the environment resolves to 1.0.7, that is usually acceptable as long as import mountainsort4 works and SpikeInterface reports mountainsort4 in installed_sorters().
Kilosort4	Install with pip install "kilosort[gui]". The Python package is kilosort, but the SpikeInterface sorter name is kilosort4. For GPU acceleration, install a PyTorch/CUDA build appropriate for the machine.
SpyKING Circus2	Used through SpikeInterface as the internal sorter spykingcircus2. It is not the same as manually installing and running the older external spyking-circus package. It should appear in SpikeInterface installed_sorters().

2.4 Optional broader analysis packages

These packages are not required for basic GUI loading, QC, sorting, metrics, reports, and curation. They are useful for broader graph/network, manifold, and machine-learning workflows.

```
pip install seaborn==0.13.2
pip install igraph==0.11.8 leidenalg==0.10.2 networkx==3.4.2
pip install optuna==3.6.1 shap==0.44.1 hdbscan==0.8.40
```

For PyTorch Geometric packages, install versions compatible with the local PyTorch, Python, CPU/GPU, and CUDA setup. Do not assume one universal command for all computers.

```
pip install torch-geometric==2.5.3 torch-scatter==2.1.2
```

2.5 Minimal practical environment for this GUI

```
spikeinterface[full,widgets]==0.103.0
mountainsort4
kilosort
probeinterface
neo
quantities
numpy
pandas
scipy
matplotlib
scikit-learn
openpyxl
xlsxwriter
h5py
joblib
tqdm
ipywidgets
ipyfilechooser
ipydatagrid
ipykernel
```

2.6 Verifying the installation

```
python -c "import spikeinterface as si; import mountainsort4; import kilosort; import probeinterface; import neo;
print('SpikeInterface', si.__version__); print('Installation OK')"
```

Expected output should include:

```
SpikeInterface 0.103.0
Installation OK
```

Then verify the Jupyter kernel:

```
jupyter kernelspec list
```

The list should include mea_nlx_sorter.

2.7 Recommended Jupyter environment check

Before running the GUI, researchers should run the following environment-check notebook cell in the same Jupyter kernel that will run the sorter UI.

A good result should show Core GUI packages: OK and installed sorters reported by SpikeInterface should include mountainsort4, kilosort4, and spykingcircus2. For Kilosort4, the check should also report kilosort package import: OK. If CUDA is available, it should show CUDA available: True.

```
# ==========================
# MEA/NLX Sorter Environment Check
# ==========================
# Run this in the same Jupyter kernel/environment that will run the sorter UI.

import sys
import os
import platform
import importlib
import importlib.util
from importlib.metadata import version, PackageNotFoundError
from pathlib import Path

import pandas as pd
```

```
# ---------------------------
# Package definitions
# ---------------------------
# package_name = pip/distribution name
# import_name  = Python import name
# target_version = version used/tested in the reference environment
# required_for_core_gui = required for core GUI/QC/sorting/metrics/reports/curation

REQUIRED_PACKAGES = [
    # Core numerical/scientific stack
    ("numpy", "numpy", "2.2.6", True),
    ("pandas", "pandas", "2.3.1", True),
    ("matplotlib", "matplotlib", "3.10.5", True),
    ("scipy", "scipy", "1.15.3", True),
    ("scikit-learn", "sklearn", "1.7.1", True),

    # Excel/output/reporting
    ("openpyxl", "openpyxl", "3.1.5", True),
    ("xlsxwriter", "xlsxwriter", "3.2.5", True),
    ("tqdm", "tqdm", "4.67.1", True),
    ("joblib", "joblib", "1.5.1", True),
    ("h5py", "h5py", "3.14.0", True),
    ("pyyaml", "yaml", "6.0.2", True),

    # SpikeInterface ecosystem
    ("spikeinterface", "spikeinterface", "0.103.0", True),
    ("mountainsort4", "mountainsort4", "1.0.6", True),
    ("probeinterface", "probeinterface", "0.3.0", True),
    ("neo", "neo", "0.14.2", True),
    ("quantities", "quantities", "0.16.2", True),

    # Jupyter UI
    ("ipywidgets", "ipywidgets", None, True),
    ("ipyfilechooser", "ipyfilechooser", None, False),
    ("ipydatagrid", "ipydatagrid", None, False),
    ("IPython", "IPython", None, True),
    ("ipykernel", "ipykernel", None, True),

    # Performance/support
    ("numba", "numba", "0.61.2", False),
    ("llvmlite", "llvmlite", "0.44.0", False),
    ("psutil", "psutil", "5.9.0", False),
    ("matplotlib-inline", "matplotlib_inline", "0.1.7", False),

    # Kilosort4
    # Important: pip package is "kilosort"; SpikeInterface sorter name is "kilosort4".
    ("kilosort", "kilosort", None, False),

    # Optional broader analysis packages
    ("seaborn", "seaborn", "0.13.2", False),
    ("igraph", "igraph", "0.11.8", False),
    ("leidenalg", "leidenalg", "0.10.2", False),
    ("networkx", "networkx", "3.4.2", False),
    ("optuna", "optuna", "3.6.1", False),
    ("shap", "shap", "0.44.1", False),
    ("hdbscan", "hdbscan", "0.8.40", False),

    # PyTorch stack, optional unless using Kilosort4/GNN workflows
    ("torch", "torch", "2.2.0", False),
    ("torch-geometric", "torch_geometric", "2.5.3", False),
    ("torch-scatter", "torch_scatter", "2.1.2", False),
]


def _get_dist_version(package_name: str):
    try:
        return version(package_name)
    except PackageNotFoundError:
        return None
    except Exception as e:
        return f"ERROR: {e}"


def _can_import(import_name: str):
    try:
        spec = importlib.util.find_spec(import_name)
        if spec is None:
            return False, "not found"
        mod = importlib.import_module(import_name)
        return True, getattr(mod, "__version__", "")
    except Exception as e:
        return False, str(e)
```

```
rows = []
for package_name, import_name, target_version, required in REQUIRED_PACKAGES:
    installed_version = _get_dist_version(package_name)
    ok_import, import_version_or_error = _can_import(import_name)

    if installed_version is None:
        status = "MISSING"
    elif not ok_import:
        status = "INSTALLED BUT IMPORT FAILS"
    elif target_version and installed_version != target_version:
        status = "VERSION DIFFERS"
    else:
        status = "OK"

    rows.append({
        "package": package_name,
        "import": import_name,
        "required_for_core_gui": bool(required),
        "target_version": target_version or "",
        "installed_version": installed_version or "",
        "import_ok": ok_import,
        "import_version_or_error": import_version_or_error,
        "status": status,
    })

pkg_df = pd.DataFrame(rows)

# ---------------------------
# System info
# ---------------------------

print("=== System / Python ===")
print(f"Python executable: {sys.executable}")
print(f"Python version:    {sys.version.split()[0]}")
print(f"Platform:          {platform.platform()}")
print(f"Working directory: {Path.cwd()}")

try:
    import psutil
    print(f"CPU logical cores: {psutil.cpu_count(logical=True)}")
    print(f"CPU physical cores:{psutil.cpu_count(logical=False)}")
    print(f"RAM total:         {psutil.virtual_memory().total / (1024**3):.1f} GiB")
except Exception as e:
    print(f"psutil system check skipped: {e}")

print("\n=== Package check ===")
display(pkg_df)

# ---------------------------
# SpikeInterface sorter checks
# ---------------------------

print("\n=== SpikeInterface sorter check ===")
try:
    import spikeinterface as si
    import spikeinterface.sorters as ss

    print(f"SpikeInterface version: {si.__version__}")

    try:
        available_sorters = ss.available_sorters()
    except Exception as e:
        available_sorters = f"ERROR: {e}"

    try:
        installed_sorters = ss.installed_sorters()
    except Exception as e:
        installed_sorters = f"ERROR: {e}"

    print(f"Available sorters reported by SpikeInterface: {available_sorters}")
    print(f"Installed sorters reported by SpikeInterface: {installed_sorters}")

    for sorter_name in ["mountainsort4", "kilosort4", "spykingcircus2"]:
        print(f"\nSorter: {sorter_name}")

        if isinstance(installed_sorters, list):
            print(f"  installed_sorters contains {sorter_name}: {sorter_name in installed_sorters}")

        # SpikeInterface 0.103.0 uses get_default_sorter_params().
        # The fallback keeps the checker tolerant to API differences.
        params = None
        param_errs = []

        for fn_name in ["get_default_sorter_params", "get_default_params"]:
```

```
            try:
                if hasattr(ss, fn_name):
                    params = getattr(ss, fn_name)(sorter_name)
                    print(f"  default params available via ss.{fn_name}: YES")
                    print(f"  first params: {list(params.keys())[:12]}")
                    break
            except Exception as e:
                param_errs.append(f"{fn_name}: {e}")

        if params is None:
            print("  default params lookup skipped/unavailable in this SpikeInterface API.")
            if param_errs:
                print("  attempted methods:")
                for err in param_errs:
                    print(f"    - {err}")

except Exception as e:
    print(f"SpikeInterface sorter check failed: {e}")

# ---------------------------
# Kilosort4 / PyTorch / CUDA check
# ---------------------------

print("\n=== Kilosort4 / PyTorch / CUDA check ===")
try:
    import torch
    print(f"torch version:          {torch.__version__}")
    print(f"CUDA available:         {torch.cuda.is_available()}")
    print(f"CUDA device count:      {torch.cuda.device_count()}")

    if torch.cuda.is_available():
        for i in range(torch.cuda.device_count()):
            print(f"CUDA device {i}:         {torch.cuda.get_device_name(i)}")
    else:
        print("CUDA note: Kilosort4 can be installed and selected, but GPU acceleration is not available in this kernel.")
except Exception as e:
    print(f"PyTorch/CUDA check failed or torch is not installed: {e}")

try:
    import kilosort
    print("kilosort package import: OK")
except Exception as e:
    print(f"kilosort package import: MISSING/ERROR: {e}")

# ---------------------------
# Tkinter check
# ---------------------------

print("\n=== Tkinter check ===")
try:
    import tkinter
    print("tkinter import: OK")
except Exception as e:
    print(f"tkinter import failed: {e}")
    print("On some systems, tkinter must be installed through conda or the OS package manager.")

# ---------------------------
# Summary and suggested fixes
# ---------------------------

missing_core = pkg_df[
    (pkg_df["required_for_core_gui"]) &
    (pkg_df["status"].isin(["MISSING", "INSTALLED BUT IMPORT FAILS"]))
]

missing_optional = pkg_df[
    (~pkg_df["required_for_core_gui"]) &
    (pkg_df["status"].isin(["MISSING", "INSTALLED BUT IMPORT FAILS"]))
]

version_diffs = pkg_df[pkg_df["status"] == "VERSION DIFFERS"]

print("\n=== Summary ===")

if missing_core.empty:
    print("Core GUI packages: OK")
else:
    print("Missing/problematic CORE GUI packages:")
    display(missing_core[["package", "import", "status", "import_version_or_error"]])

if not missing_optional.empty:
    print("\nMissing/problematic optional packages:")
    display(missing_optional[["package", "import", "status", "import_version_or_error"]])
```

```
if not version_diffs.empty:
    print("\nPackages with version differences from the reference environment:")
    display(version_diffs[["package", "target_version", "installed_version", "required_for_core_gui"]])
    print("Version differences are not automatically fatal if the package imports and the GUI runs.")

print("\nSuggested core install command if needed:")
print('pip install "spikeinterface[full,widgets]==0.103.0" mountainsort4==1.0.6 probeinterface==0.3.0 neo==0.14.2 quantities==0.16.2')
print("pip install ipywidgets ipyfilechooser ipydatagrid ipykernel numpy pandas matplotlib scipy scikit-learn openpyxl xlsxwriter tqdm
joblib h5py pyyaml")

print("\nSuggested Kilosort4 install command if needed:")
print('pip install "kilosort[gui]"')

print("\nNote: SpyKING Circus2 is checked through SpikeInterface as sorter name 'spykingcircus2'.")
```

3. High-level workflow

The GUI is organized around a stepwise workflow. The researcher first loads the core Python module, selects an input recording, performs channel-level QC, runs one sorter or a configured sorter comparison, computes outputs, and then runs two-pass unit curation. For Neuralynx data, the GUI treats each tetrode bundle as one 4-channel recording, so TT1a, TT1b, TT1c, and TT1d are loaded together as TT1 and sorted separately from TT2, TT3, and TT4.

Step	Tab/area	Purpose	Main output
1	Core loader	Load the Python core containing the backend functions used by the UI.	Core loaded status.
2	Tab 1 - Load & QC	Load MEA .h5 or Neuralynx tetrode folder, attach geometry, preprocess, compute RMS QC, select/exclude channels, and inspect traces.	Preprocessed recording, QC table, included-channel list.
3	Tab 2 - Sorting & Metrics	Choose MountainSort4, Kilosort4, or SpyKING Circus2; run sorting; compute quality metrics; export spike times; build unit reports; optionally run configured sorter comparisons.	Sorting output folder, quality_metrics.xlsx, units_spike_times.xlsx, unit_reports/, optional sorter_comparison_* folder.
4	Tab 3 - Unit Curation	Read quality_metrics.xlsx and run both curation passes in one step using editable thresholds.	Curated workbook with combined_units, kept_only, flagged_only, curation_summary, and curation_settings sheets.

Active tab layout: The old Export/Batch tab is not part of the active workflow described here. The active guide uses three tabs: Load & QC, Sorting & Metrics, and Unit Curation.

4. Core module loader

At the top of the UI, the researcher enters the path to the core .py file and clicks Load. The UI dynamically imports the core and checks that required backend functions are present. The active curation-enabled core exposes functions for loading MCS HDF5, loading Neuralynx tetrode folders, preprocessing recordings, running sorters, extracting waveforms/PCs, computing quality metrics, making reports, exporting spike times, and running two-pass curation.

Control	What it does	Researcher note
Core .py path	Path to the backend Python file.	Use the curation-enabled core file when using Tab 3.
Load button	Imports the core and binds required functions into the UI.	If functions are missing, the UI reports the missing function names.
Core status	Shows whether the import succeeded.	Sorting/QC cannot run until the core is loaded.

5. Tab 1 - Load & QC

5.1 Input type

Tab 1 supports two input modes: MEA .h5 and Neuralynx tetrode folder (.ncs). The same downstream workflow is used as much as possible, but the input handling and geometry assignment differ.

Input mode	What the researcher provides	How geometry is handled	How sorting jobs are organized
MEA .h5	One MEA HDF5 recording file plus an optional geometry CSV.	The geometry CSV maps hardware/channel IDs to x_um/y_um	One recording is sorted according to selected included

Input mode	What the researcher provides	How geometry is handled	How sorting jobs are organized
		coordinates.	channels and sorter parameters.
Neuralynx tetrode folder (.ncs)	A folder containing .ncs files such as TT1a.ncs, TT1b.ncs, TT1c.ncs, TT1d.ncs. .ntt files may be present but are not used for the tetrode scan.	The core automatically attaches a compact 4-contact tetrode geometry. Default layout is a 25 um square, whose diagonal is about 35 um.	One tetrode group is selected and sorted at a time, for example TT1, then TT2, etc.

5.2 Neuralynx tetrode scan

In Neuralynx mode, the Scan tetrodes button searches for tetrode-style .ncs filenames. It groups files by bundle name, so TT1a.ncs, TT1b.ncs, TT1c.ncs, and TT1d.ncs become the TT1 group. The dropdown shows TT1, TT2, TT3, etc., while the Members line shows the individual .ncs files in the selected group.

This grouping does not prevent excluding individual wires. The intended workflow is to load the entire tetrode bundle first, compute QC on the individual channels, then exclude broken/noisy wires using the QC table before sorting. Three retained wires can still be sorted; two retained wires are technically possible but weaker; one retained wire is no longer meaningful tetrode sorting.

5.3 Output folder and active run folder

The output folder defines where the GUI writes QC outputs, sorter folders, waveform/analyzer outputs, quality metrics, spike-time exports, reports, and curated workbooks. In Neuralynx mode, the UI creates an active run folder using the session name and selected tetrode name so that TT1, TT2, etc. do not overwrite each other.

5.4 Preprocessing settings

After loading the recording and assigning geometry, Tab 1 builds a preprocessed recording. The core applies preprocessing in this order:

```
raw recording
  -> bandpass filter
  -> optional notch filter
  -> optional common median reference (CMR)
  -> preprocessed recording used for QC and most sorters
```

Control	Meaning	Default/typical use
High-pass (Hz)	Lower cutoff for the bandpass filter.	MEA historical default may be 300 Hz; Neuralynx tetrode testing uses 500 Hz.
Low-pass (Hz)	Upper cutoff for the bandpass filter.	MEA historical default may be 6000 Hz; Neuralynx tetrode testing uses 9000 Hz.
Apply notch filter	Applies a line-noise notch, usually 60 Hz.	Recommended when 60 Hz line noise is visible.
Notch Freq (Hz)	Frequency of the notch filter.	Usually 60 Hz in US recordings.
Common median reference (CMR)	Subtracts the median across retained channels at each time point.	Can reduce shared noise. In tetrode mode, CMR is computed across the tetrode channels that remain in the recording.

5.5 RMS QC and channel selection

The QC step computes a baseline RMS for each channel from the preprocessed recording. It searches for the quietest window inside the selected baseline interval and computes RMS for each channel in that quiet segment. It also reports threshold_3x = 3 x RMS for transparency, but that value is not directly passed into MountainSort4, Kilosort4, or SpyKING Circus2 as the sorter threshold.

QC control/column	Meaning
Baseline start/end	Time interval used to search for the quietest baseline segment.
Quiet length	Length of the quiet window used for per-channel RMS calculation.
rms	Per-channel RMS in the quietest segment of the preprocessed trace.
threshold_3x	3 x RMS, useful for comparing to older threshold-crossing workflows. It is not the MS4 detect_threshold parameter.
rms_thr	Auto-flag threshold computed as mean RMS + 2 SD across channels.
auto_flag	True if a channel is above the RMS auto-flag threshold.
include	Editable decision for whether the channel is retained for sorting.

QC control/column	Meaning
quiet_start_s / quiet_end_s	The quiet window selected for that channel.

The QC table can be shown as an editable grid if ipydatagrid is available or as a fallback table with an Exclude selector. The researcher should apply edits after changing channel inclusion. Sorting uses only the channels marked include=True.

5.6 Tab 1 plotting tools

Plot tool	Input trace	What it shows	How to interpret
Flagged channel Plot 60 s	The preprocessed recording.	First 60 seconds of a selected auto-flagged channel.	Useful for seeing whether a channel is truly broken/noisy or only statistically high RMS.
Plot notch	Bandpassed trace before and after notch.	Before notch = post-bandpass; after notch = post-bandpass plus notch.	Shows whether the notch visibly reduces line noise without strongly distorting the trace.
Plot CMR	Post-notch trace before and after CMR.	Before CMR = post-bandpass + post-notch; after CMR = post-bandpass + post-notch + CMR.	Shows whether CMR removes shared artifacts or undesirably changes the channel trace.

Important: The CMR plot is intentionally post-notch on both traces. This matches the preprocessing order: bandpass first, notch second, CMR third.

6. Tab 2 - Sorting & Metrics

6.1 General sorting logic

Tab 2 is used after Tab 1 QC is complete. When the researcher clicks Run sorting, the UI first syncs any QC edits, subsets both the raw and preprocessed recordings to the included channels, and then runs the selected sorter. MountainSort4 and SpyKING Circus2 receive the preprocessed trace. Kilosort4 receives the raw channel-subset recording with optional notch applied, because Kilosort4 is designed to handle its main preprocessing internally.

```
For MountainSort4 and SpyKING Circus2:
  retained raw channels -> Tab 1 preprocessing -> sorter

For Kilosort4:
  retained raw channels -> optional notch only -> Kilosort4 internal preprocessing
```

6.2 MountainSort4 parameters

In MEA .h5 mode, the Local/Wide/Global-ish preset dropdown remains visible because spatial adjacency is important for large MEA geometries. In Neuralynx tetrode mode, the preset dropdown is hidden and the UI directly fills tetrode-specific defaults. These values remain editable.

Parameter	Meaning	MEA default behavior	Neuralynx tetrode default
detect_threshold	Detection threshold in the sorter noise-normalized detection space. Higher is stricter.	Default filled as 4.0 by the MEA preset logic.	4.0
detect_sign	Spike polarity: -1 negative, +1 positive, 0 both polarities.	0 by default.	0 by default.
adjacency_radius	Spatial radius in um used for local channel neighborhoods.	Local 200 um, Wide 400 um, Global-ish -1.	80 um. This covers the 25 um square tetrode geometry.
clip_size	Sorter waveform clip length in samples.	128 samples in the current UI.	128 samples, about 4 ms at 32 kHz.
detect_interval	Duplicate-detection lockout in samples.	10 samples, about 0.5 ms at 20 kHz.	16 samples, about 0.5 ms at 32 kHz.
ms4 workers	CPU workers used by MountainSort4.	12 by default.	12 by default; reduce on laptops.

What MountainSort4 actually receives

The core calls MountainSort4 with filter=False and whiten=True. That means Tab 1 already performed the bandpass/notch/CMR preprocessing, MountainSort4 does not bandpass again, but MountainSort4 does perform internal whitening before detection and clustering.

```
MountainSort4 input = Tab 1 preprocessed trace
Core MS4 policy:
  filter = False
  whiten = True
  detect_threshold = value from Tab 2
```

6.3 Kilosort4 parameters

Kilosort4 settings are shown as editable JSON. In MEA mode, Local/Wide/Global-ish JSON presets are available. In Neuralynx tetrode mode, the preset dropdown is hidden and the JSON is filled with 32 kHz, 4-channel tetrode-oriented values.

Kilosort4 JSON parameter	Meaning	Neuralynx tetrode default
fs	Sampling rate passed to Kilosort4.	32000
highpass_cutoff	Internal high-pass cutoff used by Kilosort4.	500 Hz
Th_universal / Th_learned	Detection/template thresholds; higher is stricter.	9 / 8
nt	Template/window length in samples.	129 samples, about 4 ms at 32 kHz.
do_CAR	Kilosort4 common-average/reference step.	true
dmin / dminx	Nominal geometry spacing.	25 um
max_channel_distance	Maximum neighborhood distance.	80 um
whitening_range / nearest_chans	Channels used for local whitening/neighborhood operations.	4 / 4
duplicate_spike_ms	Duplicate spike suppression in milliseconds.	0.5 ms
cluster_neighbors	Neighbors used during clustering.	4
skip_kilosort_preprocessing	Whether to bypass Kilosort4 preprocessing.	false in the active baseline.

Kilosort4 note: Kilosort4 is not a direct equivalent to MountainSort4. Its threshold, preprocessing, whitening, template learning, and detection model are different. The JSON makes parameters transparent and editable, but the same numeric threshold does not mean the exact same thing across sorters.

6.4 SpyKING Circus2 parameters

SpyKING Circus2 settings are also shown as editable JSON. In Neuralynx tetrode mode, the JSON is filled with 4-channel tetrode-oriented values. The active baseline uses the preprocessed trace for SpyKING Circus2, so apply_preprocessing is set to false in the tetrode JSON to avoid duplicating preprocessing unnecessarily.

SpyKING Circus2 JSON block/parameter	Meaning	Neuralynx tetrode default
apply_preprocessing	Whether SC2 performs its own preprocessing before sorting.	false
sparsity.radius_um	Spatial radius used for sparse channel neighborhoods.	80 um
filtering	Filter settings if SC2 preprocessing is enabled.	Present for transparency; normally not re-applied in current tetrode baseline.
whitening	Whitening/neighborhood settings.	Local settings suited to 4-channel tetrodes.
detection.peak_sign	Peak polarity used for detection.	both
detection.detect_threshold	SC2 detection threshold.	4.0
detection.min_peak_distance_ms	Duplicate peak suppression distance.	0.5 ms
clustering	Clustering settings for detected events.	Tetrode-local neighbor settings.

6.5 Running sorting

After Run sorting completes, the UI reports how many units were found. The sorter-specific output is written to a folder inside the active run directory. The sorting output alone is not the final curated dataset; the researcher should compute metrics, export spike times, build reports, inspect the outputs, and then run Tab 3 curation.

6.6 Post-sorting quality metrics, spike-time export, and unit reports

After sorting is complete, Tab 2 provides three additional post-sorting tools: Compute metrics, Export spike times, and Build unit reports. These steps are separate from the sorting step itself. Sorting produces the unit labels and spike trains, but the post-sorting tools convert those outputs into quantitative quality metrics, spike-time tables, and visual reports that can be reviewed during curation.

Compute metrics

The Compute metrics button calculates quality metrics for all detected units after sorting. This step requires a completed sorting run because it uses the SortingAnalyzer object generated during waveform extraction. Before metrics are calculated, the code computes templates, spike amplitudes, noise levels using a median absolute deviation approach, and, if selected, principal components. If the Include PC-based cluster metrics checkbox is enabled, principal components are computed with three components using a global-by-channel PCA mode. These PCA-derived features allow metrics such as nearest-neighbor isolation measures to be calculated when available.

The output of this step is quality_metrics.xlsx, saved in the active output directory for the current recording or tetrode. For Neuralynx tetrode mode, this means the tetrode-specific run folder. The workbook contains one row per sorted unit and includes quality metrics such as firing rate, signal-to-noise ratio, ISI violation-related metrics, amplitude cutoff, presence ratio, and any available PC/nearest-neighbor metrics.

This file is important because it becomes the main input for Tab 3: Unit Curation. Researchers should run Compute metrics before using Tab 3.

Include PC-based cluster metrics

The Include PC-based cluster metrics checkbox controls whether the program computes PCA features before calculating quality metrics. These PC-based features are used for cluster-separation metrics such as nearest-neighbor hit rate when supported by the available SpikeInterface metric set.

These metrics are useful because they evaluate whether spikes assigned to a unit occupy a consistent and separable region of waveform-feature space. A high nearest-neighbor hit rate indicates that spikes from a given unit tend to have nearby neighbors from the same unit, suggesting better cluster isolation. A low value suggests overlap with other units or noise, which may indicate contamination, poor separation, or unstable clustering.

Because PC-based metrics are more computationally expensive, the checkbox lets the researcher decide whether to compute them. For final curation, leaving this option on is recommended when runtime is acceptable.

ACG report options

Tab 2 also includes options for autocorrelogram reporting: Include ACGs (grid) and Save ACGs as multi-page PDF.

The autocorrelogram shows the distribution of spike-time intervals around each spike for a unit. It is useful for checking refractory-period structure, bursting patterns, rhythmic firing, and possible contamination. For single units, the ACG should generally show a reduced probability of spikes at very short lags, reflecting the refractory period. A strong excess of near-zero-lag spikes may indicate contamination or poor unit separation.

When Include ACGs is checked, ACG plots are included in the unit-report output. When Save ACGs as multi-page PDF is checked, the ACGs are also saved as a multi-page PDF-style report, which is easier to review for many units.

Export spike times

The Export spike times button saves the spike times for every sorted unit into units_spike_times.xlsx. This export requires that sorting has already been run. The code retrieves the recording sampling frequency from the waveform analyzer and converts each unit's spike frames into time using that sampling rate. The resulting file provides unit-by-unit spike times that can be used for downstream analyses outside the sorter GUI.

This output is useful when researchers want to compute firing rates, ISI distributions, functional connectivity, burst participation, temporal coding metrics, or custom analyses outside the GUI.

Build unit reports

The Build unit reports button creates visual per-unit reports and saves them under unit_reports/. This step uses the waveform analyzer, the current settings, and the available quality metrics. The reports are meant for human review. They provide visual summaries of each sorted unit, including waveform/template information and other diagnostic plots such as ISI or ACG-related outputs depending on the selected report options.

These reports are important because automated quality metrics should not be used alone. A unit may pass numerical thresholds but still show suspicious waveform instability, duplicated units, abnormal ISI structure, poor clustering, or noise-like activity. Conversely, some units may fail one metric for a biologically or technically explainable reason and may require manual review. The unit reports provide the visual evidence needed for that decision.

```
Recommended workflow:
Run sorting
  -> Compute metrics
  -> Export spike times
  -> Build unit reports
  -> Review metrics and reports
  -> Run Tab 3 curation
```

6.7 Compare-sorters tools

Tab 2 includes three pairwise sorter-comparison buttons: Compare MS4 vs KS4, Compare MS4 vs SC2, and Compare KS4 vs SC2. These tools run two sorters head-to-head on the same dataset and compare how similar their detected units are. This is useful when researchers want to evaluate whether a unit population is robust to sorter choice, whether a result depends strongly on one algorithm, or whether one sorter is over-splitting, under-splitting, or detecting additional questionable units.

Updated comparison workflow

In the updated UI, clicking a compare-sorters button does not immediately start the comparison. Instead, it opens a comparison setup panel inside Tab 2. This panel shows the two selected sorters side by side, with editable parameters and explanations for each sorter. The researcher reviews or modifies the parameters and then clicks Run configured comparison.

This design prevents accidental comparisons with hidden or unintended parameters. It also allows the researcher to choose different settings for each sorter before the comparison begins.

Sorter in comparison panel	Parameter interface
MountainSort4	Direct editable fields: detect_threshold, detect_sign, adjacency_radius, clip_size, detect_interval, workers.
Kilosort4	Editable JSON block with sorter-specific explanations and reset-to-default option.
SpyKING Circus2	Editable JSON block with sorter-specific explanations and reset-to-default option.

The comparison panel uses the same general parameter logic as the main sorter interface. MEA .h5 mode uses MEA-oriented geometry and sorter defaults. Neuralynx tetrode mode uses Neuralynx/tetrode-oriented defaults, including compact tetrode geometry assumptions.

Input recording used for comparison

The comparison function requires that QC has already been completed. Before running either sorter, the UI syncs the current channel-inclusion selections from the QC table and applies the same included-channel mask to both sorter inputs. This is important because the comparison should reflect sorter differences, not differences in which channels were included.

```
MS4 and SC2 receive the QC-filtered/preprocessed recording.
KS4 receives the raw channel-subset recording with optional notch applied, because KS4 normally performs its own high-
pass filtering, whitening, and preprocessing internally.
```

What the comparison pipeline generates automatically

In the updated workflow, a comparison run is not just a unit-matching step. It runs a complete mini-pipeline for each sorter being compared. For each sorter, the comparison pipeline automatically runs the sorter, exports spike times, extracts waveforms and builds the SortingAnalyzer, computes quality metrics, builds unit reports, saves the sorter settings used, compares sorter outputs, and saves agreement/matching files and a heatmap.

```
Run sorter
  -> Export spike times
  -> Extract waveforms / build analyzer
  -> Compute quality metrics
  -> Build unit reports
  -> Save sorter settings
  -> Compare sorter outputs
```

```
  -> Save agreement/matching files and heatmap
```

The waveform/analyzer folder is generated before metrics and reports because SpikeInterface quality metrics and report generation require waveform/template/analyzer information.

Comparison folder structure

Each comparison run creates a new comparison root folder in the active run directory. The folder name includes the sorter pair and a timestamp.

```
sorter_comparison_ms4_vs_ks4_YYYY-MM-DD_HH-MM-SS/
    ms4_mountainsort4/
        si_sorting/
        waveforms/
        units_spike_times.xlsx
        quality_metrics.xlsx
        unit_reports/
        sorter_settings_used.json

    ks4_kilosort4/
        si_sorting/
        waveforms/
        units_spike_times.xlsx
        quality_metrics.xlsx
        unit_reports/
        sorter_settings_used.json

    ms4_vs_ks4_matched_pairs.csv
    unmatched_ms4.csv
    unmatched_ks4.csv
    agreement_matrix_ms4_vs_ks4.csv
    agreement_heatmap_ms4_vs_ks4.png
    comparison_summary_ms4_vs_ks4.xlsx
    comparison_output_manifest.csv
```

Per-sorter outputs

Output	What it shows / why it matters
si_sorting/	SpikeInterface sorting output folder for that sorter. It stores the sorter's detected units and spike trains in a SpikeInterface-compatible format.
waveforms/	Waveform/analyzer folder generated after sorting. It contains data required for waveform templates, principal components when enabled, quality metrics, and reports.
units_spike_times.xlsx	Spike times for each detected unit from that sorter. Generated automatically during comparison.
quality_metrics.xlsx	Quality metrics for the units detected by that sorter. Generated automatically during comparison.
unit_reports/	Visual unit reports for that sorter, including waveform/template and other report plots generated by the report pipeline.
sorter_settings_used.json	Exact sorter parameters used for that sorter during the comparison. Important for reproducibility.

Cross-sorter comparison outputs

Output	What it shows / why it matters
matched_pairs.csv	Matched units between the two sorters and the agreement score when available. A matched pair means that a unit from sorter A and a unit from sorter B had enough spike-train overlap or agreement to be considered corresponding units.
unmatched_<sorter>.csv	Units detected by one sorter that did not have a matched partner in the other sorter. These are not automatically bad; they should be reviewed using reports, metrics, spike counts, waveforms, ISI structure, and biological plausibility.
agreement_matrix_<sorterA>_vs_<sorterB>.csv	Unit-by-unit agreement matrix. Rows correspond to units from sorter A and columns correspond to units from sorter B. Each value summarizes degree of agreement.
agreement_heatmap_<sorterA>_vs_<sorterB>.png	Visual heatmap of the agreement matrix. Stronger/higher values indicate stronger agreement. A clear one-to-one diagonal-like pattern suggests strong sorter agreement. Off-diagonal structure may indicate splitting, merging, or ambiguity.
comparison_summary_<sorterA>_vs_<sorterB>.xlsx	Workbook summarizing the comparison. It includes run summary, output manifest, matched pairs, unmatched units from each sorter, and the agreement matrix.
comparison_output_manifest.csv	File inventory describing what was saved, where it was saved, and what each file represents. Use this as the first file to inspect when reviewing comparison outputs.

Questions answered by the compare-sorters tool

```
Did both sorters find similar units?
Does one sorter split a unit that another sorter keeps as one cluster?
Does one sorter merge units that another sorter separates?
Does one sorter find additional units not recovered by the other?
Are the highest-quality units stable across sorting methods?
Do sorter-specific units also pass quality metrics and visual review?
```

How to interpret comparison results

Strong agreement across sorters increases confidence that a detected unit is not sorter-specific. However, sorter agreement should not be used as the only inclusion rule. A unit found by only one sorter is not automatically wrong, but it should be reviewed carefully.

Large disagreement can indicate	Examples
Signal/data issues	Noisy or unstable channels, weak waveform separation, low spike count, state-dependent or unstable firing.
Parameter issues	Different detection thresholds, different whitening/preprocessing assumptions, duplicate suppression differences.
Sorter behavior	Over-splitting by one sorter, under-splitting or merging by another sorter.
Tetrode-specific issues	Disagreement may be more common when one or more wires are noisy, when only two channels are retained, or when waveform separation is weak.

```
Recommended use:
Use compare-sorters after QC and before committing to one sorter for final analysis.
Use the comparison setup panel to confirm each sorter’s parameters before running.
Do not use sorter agreement as the only inclusion rule.
Use sorter agreement as supporting evidence alongside waveforms, ISI structure, quality metrics, PCA/cluster separation,
spike-time structure, and biological plausibility.
```

7. How sorter thresholds and whitening work

7.1 Why Tab 1 RMS is not the same as Tab 2 threshold

Tab 1 computes RMS and threshold_3x from the preprocessed trace for QC and channel review. In contrast, Tab 2 sorter thresholds are parameters passed into the selected sorting algorithm. For MountainSort4 in the active baseline, detect_threshold is applied after Tab 1 preprocessing and after MountainSort4 internal whitening. It is not a direct voltage threshold such as 4 x the RMS value shown in the Tab 1 QC table.

7.2 Conceptual example: single-channel normalization

Suppose one channel has RMS noise of 12 uV. A simple voltage detector using 4 x RMS would threshold at 48 uV. If the signal is divided by a 12 uV noise scale, a 48 uV event becomes 4 standardized noise units.

```
Raw noise RMS = 12 uV
Raw event amplitude = 48 uV
Standardized event = 48 / 12 = 4 noise units
```

This single-channel example looks similar to RMS thresholding, but real sorter whitening is multichannel and uses the covariance structure of the noise, not only one channel’s RMS.

7.3 Multichannel tetrode whitening

Tetrode whitening is more complex because the sorter sees all retained tetrode wires simultaneously. At each time point, the tetrode signal is a vector:

```
x(t) = [TT1a(t), TT1b(t), TT1c(t), TT1d(t)]
```

The background noise across the wires may be correlated. For example, TT1a and TT1b may share common noise, or all four channels may share reference noise. Whitening estimates the covariance matrix of the background signal and transforms the data so that the noise is closer to unit variance and less correlated across channels.

```
Before whitening: noise covariance may contain off-diagonal correlations. Background noise covariance might look like:
```

```
          TT1a   TT1b   TT1c   TT1d
TT1a      1.00   0.60   0.20   0.10
TT1b      0.60   1.00   0.15   0.10
TT1c      0.20   0.15   1.00   0.50
TT1d      0.10   0.10   0.50   1.00

After whitening: noise covariance is approximately identity.

          TT1a   TT1b   TT1c   TT1d
TT1a      1.00   0.00   0.00   0.00
TT1b      0.00   1.00   0.00   0.00
TT1c      0.00   0.00   1.00   0.00
TT1d      0.00   0.00   0.00   1.00

C_whitened approximately =
[[1, 0, 0, 0],
 [0, 1, 0, 0],
 [0, 0, 1, 0],
 [0, 0, 0, 1]]
```

7.4 How the covariance is calculated

Conceptually, the sorter samples background voltage from the preprocessed recording and builds a matrix X whose rows are time samples and whose columns are channels. For a tetrode, X has four columns if all four wires are retained. The covariance matrix describes how much each channel varies and how much channels vary together.

```
C = covariance(X)

For four channels:
          TT1a   TT1b   TT1c   TT1d
TT1a     var_a  cov_ab cov_ac cov_ad
TT1b     cov_ba var_b  cov_bc cov_bd
TT1c     cov_ca cov_cb var_c  cov_cd
TT1d     cov_da cov_db cov_dc var_d
```

The diagonal values are variances. The off-diagonal values are shared noise between channels. Whitening uses this covariance matrix to transform the signal into a noise-standardized space. If a channel is excluded before sorting, the covariance matrix is computed only from the retained channels.

For a two-channel example, if Channel A and Channel B each have noise SD = 10 uV and their noise correlation is 0.6, the covariance is 0.6 x 10 x 10 = 60. Variance = $SD^2 = 10^2 = 100$. The covariance matrix is [[100, 60], [60, 100]]. Whitening decomposes this covariance structure and rescales the common-noise and independent-noise directions so the whitened background has covariance close to the identity matrix.

7.5 How detect_threshold = 4 is applied after whitening

After whitening, the signal is no longer interpreted directly in microvolts. It is in a standardized noise space. A raw tetrode event might look like this before whitening:

```
TT1a = -60 uV
TT1b = -45 uV
TT1c = -20 uV
TT1d =  -8 uV
```

After whitening, the same event might become:

```
TT1a = -5.1
TT1b = -3.8
TT1c = -1.6
TT1d = -0.7
```

With detect_sign = 0 and detect_threshold = 4, this event can be detected because abs(-5.1) > 4 on one whitened channel. A different event that reaches only 3.2 standardized units may not be detected at threshold 4, even if it appears sizable in raw voltage.

Key interpretation: In the active MountainSort4 pipeline, detect_threshold = 4 means a stricter event-detection setting in MountainSort4's whitened/noise-normalized detection space. It is not 4 x the RMS column shown in Tab 1.

8. Tab 3 - Unit Curation

8.1 Purpose

Tab 3 takes the quality_metrics.xlsx file generated from Tab 2 and runs both curation passes in one step. This replaces the older manual process in which the researcher first created a flagged workbook, manually saved the kept-only sheet, fixed missing unit_id values, and then ran a second nn_hit_rate pass.

8.2 Input and output

Item	Description
Input	quality_metrics.xlsx from the active output folder. The tab can auto-find the file from the active run directory or accept a manually selected metrics workbook/CSV.
Output workbook	A curated workbook saved in the active output folder.
Important requirement	Compute metrics in Tab 2 before running Tab 3. Without quality_metrics.xlsx, Tab 3 does not have the required input.

8.3 Curation parameters

Parameter	Default	Meaning / interpretation
amplitude_cutoff <=	0.10	First-pass stability/completeness gate. NaN amplitude_cutoff values are treated as pass by default, matching the prior workflow.
presence_ratio >=	0.90	Requires the unit to be present across most of the recording. Strong and defensible for stable units, but may be strict for state-dependent or shorter valid epochs.
contamination / ISI <=	0.50	Allen-style permissive first-pass contamination threshold. Useful as an initial screen, not a substitute for visual review.
nn_hit_rate >=	0.70	Second-pass cluster-separation/isolation gate when nearest-neighbor metrics are available. Higher values are more conservative.
Treat missing NN as keep	Enabled by default	Keeps units when nn_hit_rate is unavailable rather than dropping them because PC/NN metrics were not computed.
Error if NN column missing	Optional	When enabled, curation stops if no nearest-neighbor metric column is found.

8.4 How contamination column is selected

The curation function is tolerant to different SpikeInterface quality-metric column names. It first looks for isi_violations. If that is absent, it checks isi_violations_ratio, and if that appears to be on the Allen-like scale, it uses it as the contamination metric. If neither Allen-style metric is available, it falls back to rp_contamination when present.

8.5 Output workbook sheets

Sheet	Contents
combined_units	All units with keep/flag decisions and drop reasons.
kept_only	Units that pass both enabled curation passes.
flagged_only	Units that fail at least one gate.
curation_summary	Counts and pass rates for the curation run.
curation_settings	Thresholds and options used for the run.

9. Recommended first-pass settings for Neuralynx tetrode testing

Setting	Recommended first-pass value	Rationale
Sorter	MountainSort4	Good first test for one-tetrode-at-a-time sorting.
detect_threshold	4.0, then test 3.0-4.0 if needed	Threshold affects sensitivity and unit count. Lower values recover more events but can increase contamination/splitting.
detect_sign	0	Detects both positive and negative events.
adjacency_radius	80 um	Covers all four contacts in the 25 um square tetrode geometry; 80 and 200 should behave similarly when only one tetrode is present.
detect_interval	16 samples at 32 kHz	Approximately 0.5 ms duplicate-detection lockout, matching 10 samples at 20 kHz.

Setting	Recommended first-pass value	Rationale
clip_size	128 samples at 32 kHz	Approximately 4 ms total clip duration, matching the previous 78 samples at 20 kHz.
CMR	Enabled initially, inspect CMR plot	Can reduce shared noise but should be visually checked.
notch	Enabled when 60 Hz is visible	Use Plot notch to verify benefit.

10. Practical review checklist

Stage	Checklist
Before QC	Confirm correct input mode, correct .h5 file or Neuralynx folder, correct output folder, and correct tetrode selection.
After tetrode scan	Confirm Members line shows the expected TT#a-d .ncs files. .ntt files are not used.
After QC	Review RMS table, auto-flagged channels, flagged-channel trace, notch plot, and CMR plot. Apply channel inclusion edits.
Before sorting	Confirm included channels, sorter choice, sorter parameters, and active output folder.
After sorting	Check unit count. Then compute metrics, export spike times, and build reports.
During report review	Inspect waveforms, templates, ISI/ACG structure, PCA/cluster separation, and obvious duplicate/split units.
During curation	Use Tab 3 as an automated first-pass screen, not as the only final decision. Review flagged and kept units.
During sorter comparison	Open the comparison setup panel, confirm parameters for both sorters, then run configured comparison. Review per-sorter metrics/reports plus agreement outputs.

11. Common troubleshooting notes

Issue	Likely cause / fix
Core fails to load	Use the matching curation-enabled core. The UI lists missing functions if the wrong core is loaded.
No tetrodes found	The scanner expects names like TT1a.ncs, TT1b.ncs, TT1c.ncs, TT1d.ncs. It ignores .ntt files.
Neuralynx load path error	Use an absolute folder path when possible. The core resolves paths internally, but absolute paths reduce ambiguity.
clip_size capped unexpectedly	Older UI versions used a BoundedIntText maximum of 100. Current versions allow 128 and higher.
Kilosort4 not available	Install the kilosort package and verify SpikeInterface installed_sorters() includes kilosort4. GPU acceleration depends on PyTorch/CUDA setup.
SpyKING Circus2 not available	Ensure spikeinterface[full,widgets] is installed and installed_sorters() includes spykingcircus2.
No nn_hit_rate in curation	Compute PC-based metrics in Tab 2 if nearest-neighbor metrics are needed. Otherwise, keep the default option to keep units when NN metrics are missing.
Compare-sorters creates many outputs	This is expected. Each comparison runs two complete mini-pipelines plus matching/agreement outputs.

12. Glossary

Term	Meaning
ACG	Autocorrelogram; distribution of spike intervals around each spike for one unit.
adjacency_radius	Spatial radius used by geometry-aware sorters to define local channel neighborhoods.
CMR	Common median reference; subtracts the median trace across channels at each time point.
detect_interval	Duplicate-detection lockout window in samples. It is not the biological refractory period.
detect_sign	Spike polarity setting: negative, positive, or both.
detect_threshold	Sorter-specific event detection threshold, usually in an internal noise-normalized space rather than raw uV.
SortingAnalyzer	SpikeInterface object/folder used for waveforms, templates, extensions, quality metrics, and reports.
threshold_3x	3 x RMS value reported in Tab 1 for QC transparency; not directly passed as the MS4 detect_threshold.
whitening	Noise-standardizing transformation that reduces channel-to-channel noise correlations and scales the signal relative to noise covariance.